**University of Bahrain**
College of Information Technology
Department of Computer Science

ITCS 440: Intelligent Systems
Second Semester 2025/2026

# Expert System for Mobile Phone Fault Diagnosis



| | |
|---|---|
| **Student Names** | Abdulnaser Fawzi Hamadi |
| | Youssef Gamal Elsayed |
| | Khalid Jassas Saleh |
| **Student IDs** | 202008866 |
| | 202002321 |
| | 202008213 |
| **Section** | 3 |
| **Submission Date** | May 17, 2026 |



## Introduction

Expert systems are a branch of Artificial Intelligence that emulate the decision-making ability of a human expert within a specific domain. They encode expert knowledge as a set of IF-THEN rules and use an inference engine to derive conclusions from user-provided facts. Unlike neural networks or statistical models, expert systems are fully transparent: every conclusion can be traced back to the exact rules that fired.

This project implements an expert system for diagnosing mobile phone faults. Mobile phones are complex multi-component devices (battery, screen, cellular radio, Wi-Fi, audio hardware, storage) and most users cannot easily determine whether a symptom (e.g. a black screen, no cellular signal) points to a hardware fault, a software problem, or simple user error. The system guides users through a structured symptom questionnaire and produces one or more specific diagnoses with recommended actions, replicating what an experienced repair technician would conclude.

The system is implemented in pure Python with no external libraries and runs in a command-line interface.

## System Design

### Architecture

The system follows the classic expert system architecture with three components:

1. **Knowledge Base**: A collection of 21 IF-THEN production rules covering six fault categories (power/battery, screen, network, audio, storage/software, camera) plus a

fact dictionary with human-readable descriptions of each possible diagnosis.

2. **Inference Engine**: A forward-chaining engine that iteratively applies rules to the working memory (set of known facts) until no new conclusions can be derived (fixed-point termination).

3. **Working Memory**: A dynamic set of facts populated by the user during the consultation session and extended by the inference engine as intermediate conclusions are reached.

### Knowledge Representation

Each rule is represented as a Python dictionary with three fields:

```
{
    "id":          "R1",
    "conditions": ["phone_wont_turn_on", "battery_not_charging"
       ],
    "conclusion": "faulty_battery"
}
```

Facts are stored as strings in a Python `set`, enabling O(1) membership tests during rule evaluation. The full knowledge base contains 21 rules spanning the six diagnostic categories shown in Table 1.

Table 1: Rule distribution by fault category

| Category | Rules | Rule IDs |
|---|---|---|
| Power & Battery | 5 | R1–R5 |
| Screen | 3 | R6–R8 |
| Network | 4 | R9–R12 |
| Audio | 3 | R13–R15 |
| Storage & Software | 4 | R16–R19 |
| Camera | 2 | R20–R21 |
| **Total** | **21** | |

### Inference Engine: Forward Chaining

The inference engine implements a *data-driven* forward-chaining strategy. Starting from the user-supplied symptom facts, it scans all rules on each iteration. A rule fires if and only if every condition in its `conditions` list is already present in the working memory. The conclusion is then added to the working memory. Iteration continues until a full pass over all rules produces no new conclusions (fixed-point). The algorithm is shown below.

```
def forward_chain(facts):
    fired = []
    changed = True
    while changed:
        changed = False
        for rule in RULES:
```

```
            conclusion = rule["conclusion"]
            if conclusion not in facts and \
               all(c in facts for c in rule["conditions"]):
                facts.add(conclusion)
                fired.append(rule)
                changed = True
    return fired
```

**Termination guarantee:** the knowledge base is finite and each conclusion is added at most once (the `conclusion not in facts` guard prevents re-firing). Therefore the loop terminates in at most $|Rules|$ iterations.

## Sample Rules and Knowledge Base

Table 2 lists a representative sample of ten rules from the knowledge base to illustrate the coverage and reasoning patterns of the system.

Table 2: Sample rules from the knowledge base

| ID | Conditions (IF) | Conclusion (THEN) |
|---|---|---|
| R1 | phone_wont_turn_on ∧ battery_not_charging | faulty_battery |
| R2 | phone_wont_turn_on ∧ battery_charging_ok | faulty_power_button |
| R4 | battery_drains_fast ∧ few_apps_running | replace_battery |
| R6 | screen_black ∧ phone_on | faulty_display |
| R7 | screen_cracked ∧ touch_not_responding | replace_screen |
| R9 | no_signal ∧ sim_card_present | faulty_antenna |
| R11 | wifi_not_connecting ∧ other_devices_connect_ok | faulty_wifi_module |
| R13 | no_sound ∧ headphones_plugged_in | faulty_speaker |
| R16 | phone_slow ∧ storage_full | free_up_storage |
| R17 | phone_slow ∧ storage_ok | perform_factory_reset |

Note that rules R1 and R2 share the same first condition (`phone_wont_turn_on`) but differ in the second, allowing the system to distinguish between a battery fault and a power-button fault based on a single additional observation. Similarly, R16 and R17 distinguish software corruption from a full-storage problem. This disambiguation capability is a key advantage of rule-based expert systems over simple decision trees.

## Testing and Results

### Test Scenarios

Three representative scenarios were used to validate the system.

**Scenario 1: Storage fault (Demo mode)** **Input facts:** `phone_slow`, `storage_full`, `app_keeps_crashing`

**Rules fired:**

- R16: phone_slow ∧ storage_full ⇒ **free_up_storage**
- R19: app_keeps_crashing ∧ storage_full ⇒ **free_up_storage** (already in facts, not re-added)

**Diagnosis:** "Storage is full. Delete unused files, apps, or transfer data."

**Scenario 2: Battery fault** **Input facts:** `phone_wont_turn_on`, `battery_not_charging`

**Rules fired:**

- R1: phone_wont_turn_on ∧ battery_not_charging ⇒ **faulty_battery**

**Diagnosis:** "The battery is defective and must be replaced by a technician."

**Scenario 3: Network fault** **Input facts:** `no_signal`, `sim_card_present`

**Rules fired:**

- R9: no_signal ∧ sim_card_present ⇒ **faulty_antenna**

**Diagnosis:** "The cellular antenna is damaged. Hardware repair required."

### System Behaviour

All three scenarios produced correct, expected diagnoses. The system correctly handles multiple simultaneous faults: if the user reports both battery and screen symptoms, both diagnoses are returned independently. The fixed-point loop ensures that no applicable rule is missed even when a rule's conclusion acts as a condition for another rule (chained inference).

When no rule conditions are satisfied, the system outputs a message directing the user to a technician rather than producing a wrong or empty diagnosis silently.

## Conclusion

This project successfully demonstrates the core principles of rule-based expert systems: knowledge representation as IF-THEN rules, forward-chaining inference, and transparent explainability. The mobile phone fault diagnosis domain provided a natural, practically relevant use case with clear symptom-to-cause mappings that translate cleanly into production rules.

The system covers 21 rules across six hardware/software categories and correctly disambiguates between fault types that share common symptoms (e.g. distinguishing a dead battery from a broken power button). All diagnosed conclusions are accompanied by plain-language repair advice.

Future extensions could include: a graphical user interface, probabilistic confidence scores for uncertain symptoms, a larger knowledge base contributed by actual repair technicians, and backward chaining to ask only the most relevant questions for a given hypothesis.